

"""

bias_classifier.py

Three drop-in classifiers, one interface.

```
from bias_classifier import SemanticClassifier
from bias_classifier import HybridClassifier
from bias_classifier import LLMClassifier
```

Each exposes:

```
classifier.classify(text: str) -> tuple[str, float]
# returns (bias_label, confidence 0-1)
```

Drop any one into bayesian_middleware.py:

```
from bias_classifier import HybridClassifier as BiasClassifier
...
bias, conf = BiasClassifier().classify(user_message)
```

BIAS LABELS (shared across all classifiers)

anchoring	- stated fact, probably wrong, treated as ground truth
vague_intent	- insufficient signal to act on
confirmation_bias	- wants validation, not truth
dunning_kruger	- confident beyond competence
framing_effect	- question loaded with implicit assumption
clear_intent	- rare; user actually knows what they want
unknown	- no signal detected

"""

```
from __future__ import annotations
import re
import json
import math
import time
import requests
from typing import Optional
```

— Config —

```
OLLAMA_BASE = "http://localhost:11434"
EMBED_MODEL = "nomic-embed-text"
LLM_MODEL   = "mistral"           # any instruction-tuned model you have
```

```

BIAS_LABELS = [
    "anchoring",
    "vague_intent",
    "confirmation_bias",
    "dunning_kruger",
    "framing_effect",
    "clear_intent",
    "unknown",
]

# -----
# CLASSIFIER 1 – SEMANTIC (embedding cosine similarity)
# -----

# Exemplar sentences per bias class.
# These are embedded once at init and cached.
# Add more exemplars to improve accuracy – quality beats quantity.
_EXEMPLARS: dict[str, list[str]] = {
    "anchoring": [
        "The answer is definitely 42.",
        "It has to be Python, everyone uses Python.",
        "I know for a fact it's caused by X.",
        "The price should be around £500, that's the standard.",
        "It must be a memory leak, I'm sure of it.",
        "The correct approach is obviously microservices.",
    ],
    "vague_intent": [
        "Just make something good.",
        "I don't know, whatever works.",
        "Something about data, maybe?",
        "Can you just do it?",
        "Make it better somehow.",
        "I want a thing that does stuff.",
        "Yeah just sort it out.",
    ],
    "confirmation_bias": [
        "You agree with me right?",
        "I knew it was their fault all along.",
        "Prove that my approach is correct.",
        "Tell me I'm right about this.",
        "Everyone agrees this is the best way.",
        "I thought so – it is their fault.",
        "Can you confirm my analysis is correct?",
    ],
    "dunning_kruger": [
        "This is obviously simple, I don't understand why people struggle.",
        "I basically already know how this works.",
    ],
}

```

```

        "It's a no-brainer, clearly the answer is X.",
        "Everyone knows this is how you do it.",
        "I'm pretty much an expert at this.",
        "How hard can it be? Just do X.",
        "I've read one article so I understand the whole field now.",
    ],
    "framing_effect": [
        "Why is this technology so terrible?",
        "Shouldn't you always use tabs instead of spaces?",
        "Why don't people just follow best practices?",
        "Isn't it obvious that approach A is worse than B?",
        "Why is everyone so bad at this?",
        "Shouldn't the system just work without configuration?",
    ],
    "clear_intent": [
        "I need a Python function that takes a list and returns the top 3 items b",
        "Write a bash script that backs up /etc every Sunday at 2am.",
        "Explain the difference between TCP and UDP in two paragraphs.",
        "Given this JSON schema, generate a validator.",
        "Refactor this function to use async/await, keeping the same interface.",
        "I want exactly three options, each under 50 words.",
    ],
    "unknown": [
        "Hello.",
        "Thanks.",
        "What time is it?",
        "Can you help me?",
        "Interesting.",
    ],
    ],
}

```

```

def _cosine(a: list[float], b: list[float]) -> float:
    dot = sum(x * y for x, y in zip(a, b))
    na = math.sqrt(sum(x * x for x in a))
    nb = math.sqrt(sum(x * x for x in b))
    if na == 0 or nb == 0:
        return 0.0
    return dot / (na * nb)

```

```

def _embed(text: str) -> list[float]:
    resp = requests.post(
        f"{OLLAMA_BASE}/api/embeddings",
        json={"model": EMBED_MODEL, "prompt": text},
        timeout=15,
    )

```

```

resp.raise_for_status()
return resp.json()["embedding"]

```

```

class SemanticClassifier:

```

```

    """
    Embeds exemplars once at init, then cosine-sims each input.
    Fast at inference time; slow-ish first call (embedding all exemplars).

    Confidence = normalised gap between top-1 and top-2 similarity scores.
    """

```

```

    def __init__(self, warm: bool = True):

```

```

        # centroid_cache: bias_label -> mean embedding vector
        self._centroids: dict[str, list[float]] = {}
        if warm:
            self._build_centroids()

```

```

    def _build_centroids(self):

```

```

        print("[SemanticClassifier] Building exemplar centroids ...")
        for label, sentences in _EXEMPLARS.items():
            vecs = [_embed(s) for s in sentences]
            dim = len(vecs[0])
            centroid = [
                sum(v[i] for v in vecs) / len(vecs)
                for i in range(dim)
            ]
            self._centroids[label] = centroid
        print("[SemanticClassifier] Ready.")

```

```

    def classify(self, text: str) -> tuple[str, float]:

```

```

        if not self._centroids:
            self._build_centroids()

        vec = _embed(text)
        scores = {
            label: _cosine(vec, centroid)
            for label, centroid in self._centroids.items()
        }

```

```

        ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)
        top_label, top_score = ranked[0]
        second_score = ranked[1][1] if len(ranked) > 1 else 0.0

```

```

        # Confidence = how much top-1 beats top-2, normalised
        gap = top_score - second_score
        confidence = min(0.97, max(0.03, gap * 5)) # scale: 0.2 gap -> 1.0 conf

```

```

        return top_label, round(confidence, 3)

# -----
# CLASSIFIER 2 - HYBRID (semantic + keyword, weighted)
# -----

_KEYWORD_PATTERNS: dict[str, list[str]] = {
    "confirmation_bias": [
        r"\bprove\b", r"\bconfirm\b", r"\byou agree\b", r"\bbright\?\s*$",
        r"\bi knew\b", r"\bi thought so\b", r"\btold you\b",
    ],
    "dunning_kruger": [
        r"\bobviously\b", r"\bclearly\b", r"\beveryone knows\b",
        r"\bno[- ]brainer\b", r"\bi'?m.{0,10}expert\b",
        r"\bhow hard can\b", r"\bjust do\b",
    ],
    "anchoring": [
        r"\bthe answer is\b", r"\bit'?s definitely\b", r"\bhas to be\b",
        r"\bmust be\b", r"\bi know it'?s\b", r"\bfor a fact\b",
    ],
    "framing_effect": [
        r"\bwhy (is|are|don'?t|doesn'?t|can'?t)\b",
        r"\bshouldn'?t.{0,20}always\b",
        r"\bisn'?t it obvious\b",
        r"\bwhy is .{0,20}so\b",
    ],
    "vague_intent": [
        r"^.{0,20}$",
        r"\bsomething\b.{0,15}\bgood\b",
        r"\bwhatever\b", r"\bidunno\b", r"\bi don'?t know\b",
        r"\bjust.{0,10}(do|make|give|sort)\b",
        r"\bsomething about\b",
    ],
    "clear_intent": [
        r"\bspecifically\b", r"\bexactly\b",
        r"\b(the )?(format|output|structure|interface|signature)\b",
        r"\bstep[- ]by[- ]step\b",
        r"\bgiven.{0,30}(constraint|requirement|schema)\b",
        r"\bkeeping the same\b", r"\bwithout changing\b",
    ],
}

# How much weight each layer gets (must sum to 1.0)
_SEMANTIC_WEIGHT = 0.65
_KEYWORD_WEIGHT = 0.35

```

```

def _keyword_scores(text: str) -> dict[str, float]:
    """Returns raw [0,1] score per label from keyword hits."""
    text_l = text.lower()
    scores: dict[str, float] = {label: 0.0 for label in BIAS_LABELS}
    for label, patterns in _KEYWORD_PATTERNS.items():
        hits = sum(1 for p in patterns if re.search(p, text_l))
        scores[label] = min(1.0, hits / max(1, len(patterns)) * 3)
    return scores


class HybridClassifier:
    """
    Weighted combination of SemanticClassifier and keyword heuristics.
    Falls back gracefully if Ollama is unreachable (keyword-only mode).

    semantic_weight + keyword_weight = 1.0
    Tune in _SEMANTIC_WEIGHT / _KEYWORD_WEIGHT above.
    """

    def __init__(
        self,
        semantic_weight: float = _SEMANTIC_WEIGHT,
        keyword_weight: float = _KEYWORD_WEIGHT,
        warm: bool = True,
    ):
        self.sw = semantic_weight
        self.kw = keyword_weight
        self._semantic: Optional[SemanticClassifier] = None
        self._ollama_ok = True
        if warm:
            self._init_semantic()

    def _init_semantic(self):
        try:
            self._semantic = SemanticClassifier(warm=True)
        except Exception as e:
            print(f"[HybridClassifier] Ollama unreachable ({e}). Keyword-only mod")
            self._ollama_ok = False

    def classify(self, text: str) -> tuple[str, float]:
        kw_scores = _keyword_scores(text)

        if not self._ollama_ok or self._semantic is None:
            # Pure keyword fallback
            best = max(kw_scores, key=kw_scores.get)

```

```

        score = kw_scores[best]
        return (best if score > 0 else "unknown"), round(min(0.85, score), 3)

# Semantic scores (normalise cosine sims to [0,1])
sem_label, sem_conf = self._semantic.classify(text)
sem_scores = {label: 0.0 for label in BIAS_LABELS}
sem_scores[sem_label] = sem_conf

# Weighted fusion
fused = {}
for label in BIAS_LABELS:
    fused[label] = (
        self.sw * sem_scores[label] +
        self.kw * kw_scores[label]
    )

ranked      = sorted(fused.items(), key=lambda x: x[1], reverse=True)
top_label   = ranked[0][0]
top_score   = ranked[0][1]
second      = ranked[1][1] if len(ranked) > 1 else 0.0

# Agreement bonus: if both layers agree, boost confidence
agree_bonus = 0.10 if sem_label == max(kw_scores, key=kw_scores.get) else
confidence   = min(0.97, top_score + agree_bonus)

return top_label, round(confidence, 3)

```

```

# _____
# CLASSIFIER 3 – LLM-AS-CLASSIFIER (structured JSON from Ollama)
# _____

```

```

_LLM_SYSTEM = """\
You are a cognitive bias classifier. Given a user message, identify which \
cognitive bias or intent pattern it most strongly exhibits.

```

Choose EXACTLY ONE label from this list:

anchoring	- states a "fact" as ground truth, probably wrong
vague_intent	- insufficient detail to act on
confirmation_bias	- seeking validation rather than truth
dunning_kruger	- confident beyond demonstrated competence
framing_effect	- question contains a loaded or misleading assumption
clear_intent	- specific, well-scoped, actionable request
unknown	- no detectable pattern

Respond ONLY with a JSON object, no markdown, no explanation:

```

{"label": "<label>", "confidence": <0.0-1.0>, "reason": "<one sentence>"}

```

```
"""
```

```
class LLMClassifier:
```

```
    """
```

```
    Sends user input to a local Ollama model and parses structured JSON output.  
    Slowest of the three (~1 extra LLM call per user turn).  
    Most interpretable - `reason` field explains the classification.
```

```
    Falls back to keyword classifier if Ollama is unreachable or JSON is malformed  
    """
```

```
def __init__(self, model: str = LLM_MODEL, timeout: int = 20):  
    self.model = model  
    self.timeout = timeout  
    self._fallback = None # lazy-init keyword fallback
```

```
def classify(self, text: str) -> tuple[str, float]:
```

```
    try:  
        result = self._call_llm(text)  
        label = result.get("label", "unknown")  
        conf = float(result.get("confidence", 0.5))  
        reason = result.get("reason", "")
```

```
        if label not in BIAS_LABELS:  
            label = "unknown"  
            conf = 0.30
```

```
        # Store reason for optional inspection  
        self.last_reason = reason  
        return label, round(min(0.97, conf), 3)
```

```
    except Exception as e:  
        print(f"[LLMClassifier] Failed ({e}), falling back to keywords.")  
        return self._keyword_fallback(text)
```

```
def _call_llm(self, text: str) -> dict:
```

```
    resp = requests.post(  
        f"{OLLAMA_BASE}/api/chat",  
        json={  
            "model": self.model,  
            "stream": False,  
            "messages": [  
                {"role": "system", "content": _LLM_SYSTEM},  
                {"role": "user", "content": text},  
            ],  
            "options": {
```



```

        "temperature": 0.1,    # low temp for consistent classificatio
        "num_predict": 120,
    },
    },
    timeout=self.timeout,
)
resp.raise_for_status()
raw = resp.json()["message"]["content"].strip()

# Strip accidental markdown fences
raw = re.sub(r"^```[a-z]*\n?", "", raw)
raw = re.sub(r"\n?```$", "", raw)

return json.loads(raw)

def _keyword_fallback(self, text: str) -> tuple[str, float]:
    if self._fallback is None:
        self._fallback = HybridClassifier(warm=False)
    return self._fallback.classify(text)

@property
def last_reason(self) -> str:
    return getattr(self, "_last_reason", "")

@last_reason.setter
def last_reason(self, v: str):
    self._last_reason = v

# -----
# QUICK BENCHMARK — run this file directly
# -----

_TEST_CASES = [
    ("The answer is definitely TCP, I'm sure of it.", "anchoring"),
    ("Just make something good, I don't know.", "vague_intent"),
    ("Tell me I'm right that Python is the best language.", "confirmation_bias"),
    ("This is obviously simple, how hard can it be?", "dunning_kruger"),
    ("Why is everyone so bad at writing documentation?", "framing_effect"),
    ("Write a regex that matches UK postcodes exactly.", "clear_intent"),
    ("Thanks.", "unknown"),
]

def benchmark(classifier, name: str):
    print(f"\n— {name} —")
    correct = 0

```

```

for text, expected in _TEST_CASES:
    t0 = time.perf_counter()
    label, conf = classifier.classify(text)
    elapsed = time.perf_counter() - t0
    ok = "✓" if label == expected else "✗"
    correct += label == expected
    reason = ""
    if hasattr(classifier, "last_reason") and classifier.last_reason:
        reason = f" [{classifier.last_reason}]"
    print(f" {ok} {label:<22} conf={conf:.2f} {elapsed:.2f}s | expected={e
print(f" Accuracy: {correct}/{len(_TEST_CASES)}")

if __name__ == "__main__":
    import sys
    mode = sys.argv[1] if len(sys.argv) > 1 else "hybrid"

    if mode == "semantic":
        benchmark(SemanticClassifier(), "SEMANTIC")
    elif mode == "llm":
        benchmark(LLMClassifier(), "LLM-AS-CLASSIFIER")
    elif mode == "all":
        benchmark(HybridClassifier(), "HYBRID")
        benchmark(LLMClassifier(), "LLM-AS-CLASSIFIER")
        # Semantic shares warm cache with hybrid so run last
        benchmark(SemanticClassifier(), "SEMANTIC")
    else:
        benchmark(HybridClassifier(), "HYBRID")

```